

User guide polyN framework

Clément Nober

Aout 2024

1 Introduction

This framework has been created to calibrate the polyN minimalist function on sheet metals.

$$\bar{\sigma}_{polyN}^2 = [P_N^{2D}(\sigma_x - \sigma_z, \sigma_y - \sigma_z, \sigma_{xy})]^2 + k_{xz}\sigma_{xz}^2 + k_{yz}\sigma_{yz}^2 \quad (1)$$

This framework allows to :

1. Calibrate on experimental data
2. Calibrate on large strain tests and simulations.
3. Plot some useful graphs

Libraries required : numpy, pandas, scipy, scikit-learn, matplotlib.pyplot

2 Guide

2.1 Experiments results formatting

You have to copy paste the "polyN_optimization" directory wherever you want. In the same directory as you put the "polyN_optimization" directory, create a "lab_data" folder. In this folder, put your experiments results in a folder named "{material}_results" (replace {material} by the name of the studied material). In this folder, please put all your experiments in a folder "DATA". Your experiments have to be named under this format : "{test}_{ori}_{N}.csv" (ex : UT_00_1.csv). You can also add other files but there must be a "_" in front of the filename to prevent errors. Note also that experiments other than : UT, NT6, NT20, SH, CH should not be included, it could cause some errors in the large strain simulations. Then the experiments results will be copied in the "polyN_optimization/results_exp" directory when you will perform.

```
↪ lab_data
    ↪ {material}_results
        ↪ DATA
            ↪ UT_00_1.csv
            ↪ UT_00_2.csv
            ↪ SH_000_1.csv
            ↪ _YLD2000_pre.csv
↪ polyN_optimization
    ↪ results_exp
        ↪ {material}
            ↪ {where files will be pasted}
```

2.2 Parameters file : param.txt

This file is one of the most important files in the whole framework. That is basically where you will be able to choose the degree of your polyN, if you want to use protomodel points, the weights etc. Here are the parameters explained :

1. material : string, material studied (ex : DP700)
2. enu : float, poisson ratio
3. density : float
4. degree : int, polyN degree
5. law* : string, hardening law
6. protomodel : string, points used to increased the size of the dataset (bezier or mises)
7. nb_virtual_pt : int, number of points added to the dataset from protomodel
8. s_seg_dir : float, parameter to build the directionnal data segment in the bezier model, must be in [0.0, 1.0]

9. `s.section` : float, parameter to build section's curves in the bezier model, must be in $[0.0, 1.0]$
10. `nt6` : 0 or 1, to include plane stress points to the data set. If 1, the algorithm will proceed to a first optimization and then add the pst points to the model iteratively for `n_opti` times.
11. `n_opti` : int
12. `sh` : 0 or 1, to include shear points to the data set.
13. `weight_ut` : float (in $[0.0, 1.0]$), weight for stress points (compared to r-values, true the name is not semantically correct)
14. `weight_exp` : float (in $[0.0, 1.0]$), weight for experimental points (ut points))
15. `weight_e2` : float (in $[0.0, 1.0]$), weight for pst and shear points if added (keep to 0, and the weight for virtual points will be $1 - \text{weight_exp}$)
16. `input_type*` : string, "UMAT" or "VUMAT"
17. `var_optim` : format : int,int,int,..., list of the variables to optimize in the framework. (The first variable is 0 and is fixed to the value 1, so the variables must be superior to 1)
18. `opti` : 0 or 1, variable to delete
19. `loadcoeff` : to delete
20. `gseed` : to delete

2.3 Optimizing on experimental data

Once you have modified the `param.txt` file, you can run the `optimize_polyN_mini.py`. In the `coeff/material` directory 2 files will appear : `{material}_poly{degree}_mini.npy`, `{material}_{law}.npy` containing respectively the coefficients for the polyN function and the 3 parameters for the law given (works for Voce and Swift).

To see how the experimental data is processed you can refer to `get_calibration_data.py` file.

To see how the optimization procedure, please refer to `optimize_polyN_mini.py` and the following articles : [Soa23] [SD23], [SYC08]

2.4 Optimizing on large strain simulations

Once you have modified the `param.txt` file, you can copy the whole polyN directory in the cluster. Then get in the directory on the cluster terminal and run the following command :

```
sbatch -C ib -n 1 -mem-per-cpu=16384 -t 0-24 -wrap="python framework_cluster.py"
```

The algorithm will run simulations and save the optimized coefficients in a file `{material}_poly{degree}_mini.p.npy` with p representing the variables optimized (a 0 separate each variable ex : 106 means 1 and 6 optimized).

To see how simulations are runned please have a look at `run_large_sim_cluster.py`. The subroutine `UMAT_polyN_mini_swift.for` associated with the abaqus files has been tested on 0, 45 and 90 degree direction. To have details on the return mapping algorithm, please have a look to this AppendixB of this article : [SB11]

2.5 Plots

All the plots on the presentation come from the `check/plot_ys_rval.py` file except the ones for bezier which come from `bezier.py`.

References

- [SYC08] Stefan Soare, Jeong Whan Yoon, and Oana Cazacu. "On the use of homogeneous polynomials to develop anisotropic yield functions with applications to sheet forming". In: *International Journal of Plasticity* 24.6 (2008), pp. 915–944. ISSN: 0749-6419. DOI: <https://doi.org/10.1016/j.ijplas.2007.07.016>. URL: <https://www.sciencedirect.com/science/article/pii/S0749641907001118>.
- [SB11] Stefan C. Soare and Frédéric Barlat. "A study of the Yld2004 yield function and one extension in polynomial form: A new implementation algorithm, modeling range, and earing predictions for aluminum alloy sheets". In: *European Journal of Mechanics - A/Solids* 30.6 (2011), pp. 807–819. ISSN: 0997-7538. DOI: <https://doi.org/10.1016/j.euromechsol.2011.05.006>. URL: <https://www.sciencedirect.com/science/article/pii/S0997753811000726>.
- [SD23] Stefan Soare and Martin Diehl. "Calibration and fast evaluation algorithms for homogeneous orthotropic polynomial yield functions". In: (Mar. 2023). DOI: 10.13140/RG.2.2.16581.32488.
- [Soa23] Stefan C. Soare. "Bezier5YS and SHYqp: A general framework for generating data and for modeling symmetric and asymmetric orthotropic yield surfaces". In: *European Journal of Mechanics - A/Solids* 97 (2023), p. 104781. ISSN: 0997-7538. DOI: <https://doi.org/10.1016/j.euromechsol.2022.104781>. URL: <https://www.sciencedirect.com/science/article/pii/S0997753822002133>.